

Object oriented Programming | SAQ

1. Describe the main features of object-oriented programming (OOP).

| Main Features of Object-Oriented Programming (OOP)

The main features of OOP are:

1. **Encapsulation** – Combines **data and methods** into a single unit called a **class** and protects the data from direct access.
2. **Abstraction** – Shows only the **essential information** and hides unnecessary implementation details.
3. **Inheritance** – Allows a new class to **reuse the properties and methods** of an existing class.
4. **Polymorphism** – Allows the **same method or operation to perform different tasks** depending on the situation.
5. **Class and Object** – A class is a blueprint for creating objects, while an object is an instance of a class.

2. Write your opinion in details on the following statement. "Java is NOT a pure language."

The statement “**Java is NOT a pure language**” is correct because **Java supports both Object-Oriented Programming (OOP) and non-OOP features**.

| Reasons

1. Supports OOP Features

Java supports major OOP features such as:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Classes and Objects

2. Supports Non-OOP Features

Java also supports some non-OOP features, such as **primitive data types** (`int` , `char` , `float` , `double` , `boolean`) and **static methods**.

3. Primitive Data Types are Not Objects

In Java, primitive data types are not objects.

```
int age = 20;
```

4. Static Methods Can Work Without Objects

Static methods belong to the class and can be called without creating an object.

```
Math.sqrt(25);
```

Conclusion

Therefore, Java is not a pure OOP language because it supports both OOP and non-OOP features. However, Java is mainly designed as an object-oriented programming language.

3. Define program and instructions.

Program

A program is a set of instructions given to a computer to perform a specific task.

Instruction

An instruction is a single command that tells a computer to perform a specific operation.

4. illuminate the data types in java with proper example.

Two Main Types of Data in Java

Java data can be mainly divided into two types:

1. Primitive Data Types

Primitive data types are the basic data types in Java. They store simple values directly.

Examples: `int`, `float`, `char`, `boolean`

```
int age = 20;  
char grade = 'A';
```

2. Classes

A class is a user-defined data type that contains data and methods. Objects are created from classes.

Examples: `Student`, `Employee`

```
class Student {  
    String name;  
    int age;  
}  
  
Student s1 = new Student();
```

Here, `Student` is a class and `s1` is an object of that class.

Short Difference

Primitive Data Types	Classes
Store simple values.	Define objects.
Built into Java.	Created by the programmer.
Example: <code>int</code> , <code>char</code>	Example: <code>Student</code> , <code>Employee</code>

5. Give an example of creating an object and method in java

Example of Creating an Object and Method in Java

```
class Student {  
  
    // Method  
    void display() {  
        System.out.println("Hello, Student!");  
    }  
  
    public static void main(String[] args) {  
  
        // Creating an object  
        Student s1 = new Student();  
  
        // Calling the method  
        s1.display();  
    }  
}
```

Explanation

- **Student** → Class
- **display()** → Method
- **Student s1** → Object reference
- **new Student()** → Creates the object
- **s1.display()** → Calls the method

Output:

```
Hello, Student!
```

6. Explain the rules of constructor and the types Of constructor in java

Constructor in Java

A constructor is a special member of a class that is used to initialize an object. It is automatically called when an object is created.

Rules of Constructor

1. Its name must be the same as the class name.
2. It has no return type, not even `void`.
3. It is automatically called when an object is created.
4. A class can have more than one constructor.

Types of Constructor

1. Default Constructor

A constructor with no parameters.

```
Student() { }
```

2. Parameterized Constructor

A constructor with one or more parameters.

```
Student(String name) { }
```

7. Differentiate between array and array-list.

Array	ArrayList
Size is fixed.	Size is dynamic and can change.
Can store primitive and object data.	Stores objects only (primitive types use wrapper classes).
Faster and simpler.	Provides more built-in methods.
Length is found using <code>length</code> .	Size is found using <code>size()</code> .
Example: <code>int[] a = new int[5];</code>	Example: <code>ArrayList<Integer> a = new ArrayList<>();</code>

8. Provide an example Of constructor overloading with proper explanation.

Constructor Overloading in Java

Constructor overloading means having more than one constructor in the same class with different parameters.

Example

```
class Student {
    String name;
    int age;

    // Constructor 1
    Student() {
        name = "Unknown";
        age = 0;
    }

    // Constructor 2
    Student(String n) {
        name = n;
        age = 0;
    }

    // Constructor 3
    Student(String n, int a) {
        name = n;
        age = a;
    }

    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student("Rahim");
        Student s3 = new Student("Karim", 20);

        System.out.println(s1.name + " " + s1.age);
        System.out.println(s2.name + " " + s2.age);
        System.out.println(s3.name + " " + s3.age);
    }
}
```

Explanation

Here, the Student class has three constructors:

- **Student()** → takes no parameter.
- **Student(String n)** → takes one parameter.
- **Student(String n, int a)** → takes two parameters.

Since all constructors have the same name (**Student**) but different parameters, this is called **constructor overloading**.

9. Write short notes on for-each loop and for loop.

Short Notes on for Loop and for-each Loop

1. for Loop

The for loop is used to repeat a block of code a specific number of times. It is useful when we know the number of iterations.

Example:

```
for (int i = 1; i ≤ 5; i++) {  
    System.out.println(i);  
}
```

2. for-each Loop

The for-each loop is used to access each element of an array or collection one by one. It is simpler than the normal for loop.

Example:

```
int[] numbers = {10, 20, 30};  
  
for (int n : numbers) {  
    System.out.println(n);  
}
```

Main Difference

- **for loop:** Uses an index and gives more control over the loop.
- **for-each loop:** Directly accesses each element and is easier to use.

10. Illustrate the concept of method overloading with proper example. why we use method overloading

Method Overloading

Method overloading means having two or more methods with the same name but different parameters in the same class. It is used to perform similar tasks in different ways.

Example

```
class Calculator {  
  
    // Method with two int parameters  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method with three int parameters  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
  
        System.out.println(c.add(10, 20));  
        System.out.println(c.add(10, 20, 30));  
    }  
}
```

Explanation

Here, the `add()` method is used twice:

- `add(int a, int b)` → takes 2 parameters
- `add(int a, int b, int c)` → takes 3 parameters

Both methods have the same name but different parameters, so this is called **method overloading**.

Why Do We Use Method Overloading?

1. It increases the readability of the program.
2. It allows us to use the same method name for similar tasks.
3. It makes the program easier to understand and use.

1. Write Once, Run Everywhere” in Java

Write Once, Run Everywhere (WORA) means a Java program can be written once and run on different operating systems.

- Java source code is converted into bytecode by the compiler.
- Bytecode can run on any system that has a JVM.
- Each operating system has its own JVM.
- Therefore, the same Java program can run on Windows, Linux, and macOS without changing the code.

2. Write your own opinion for the ways on breaking the barriers of structured programming methodology.

Ways to Break the Barriers of Structured Programming

In my opinion, the limitations of structured programming can be overcome by using **Object-Oriented Programming (OOP)**.

1. **Use Classes and Objects** – Organize programs using classes and objects instead of only functions.
2. **Use Encapsulation** – Combine data and methods together and protect data from direct access.
3. **Use Inheritance** – Reuse existing code by creating new classes from existing classes.
4. **Use Polymorphism** – Allow the same method to perform different tasks.
5. **Use Abstraction** – Hide unnecessary details and show only important information.

Conclusion: OOP provides reusability, security, flexibility, and easier maintenance, helping overcome the barriers of structured programming.

4. Define Object, Describe the characteristics of object with necessary figure/example.

Definition

An object is an instance of a class. It represents a real-world entity and contains data and methods.

Characteristics of an Object

An object mainly has three characteristics:

1. **State** – Represents the data or properties of an object.
Example: A student has name, age, and marks.
2. **Behavior** – Represents the actions or methods performed by an object.
Example: A student can `study()` and `display()`.
3. **Identity** – A unique identity that distinguishes one object from another.

Example

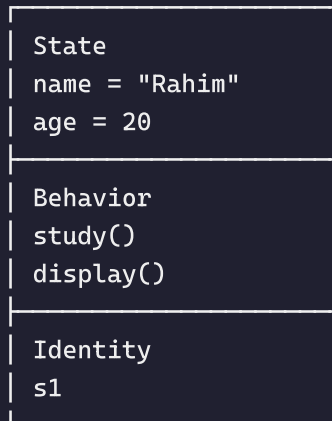
```
class Student {
    String name; // State
    int age; // State

    void study() { // Behavior
        System.out.println("Student is studying");
    }
}

Student s1 = new Student(); // Object
```


Figure

OBJECT: Student



In short: An object has Identity + State + Behavior.

4. Give proper example of various types of access modifier in java.

Access Modifiers in Java

Access modifiers control the accessibility of classes, variables, methods, and constructors in Java.

There are four types of access modifiers:

Access Modifier	Access Level
<code>public</code>	Accessible from everywhere
<code>private</code>	Accessible only within the same class
<code>protected</code>	Accessible within the same package and subclasses
Default	Accessible within the same package

1. public

A public member can be accessed from anywhere.

```
class Student {
    public String name = "Rahim";

    public void display() {
        System.out.println(name);
    }
}
```

2. private

A private member can be accessed only inside the same class.

```
class Student {  
    private int age = 20;  
  
    void display() {  
        System.out.println(age);  
    }  
}
```

3. protected

A protected member can be accessed within the same package and by subclasses.

```
class Student {  
    protected int marks = 80;  
}
```

4. Default

When no access modifier is written, it is called default access. It can be accessed within the same package.

```
class Student {  
    int age = 20; // Default  
}
```

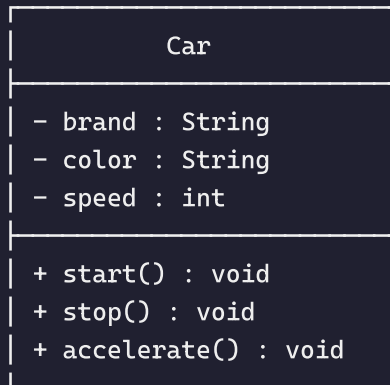
In Short

```
public → Everywhere  
protected → Same package + Subclass  
default → Same package  
private → Same class
```

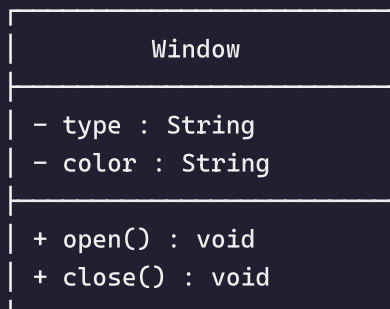
5. Draw a Unified Modeling Language (UML) representation on the Car and Window class.

UML Representation of Car and Window Classes

1. Car Class



2. Window Class



6. Analyze the Banking Software using Object-oriented analysis and find their inter-connection

Banking Software Using Object-Oriented Analysis

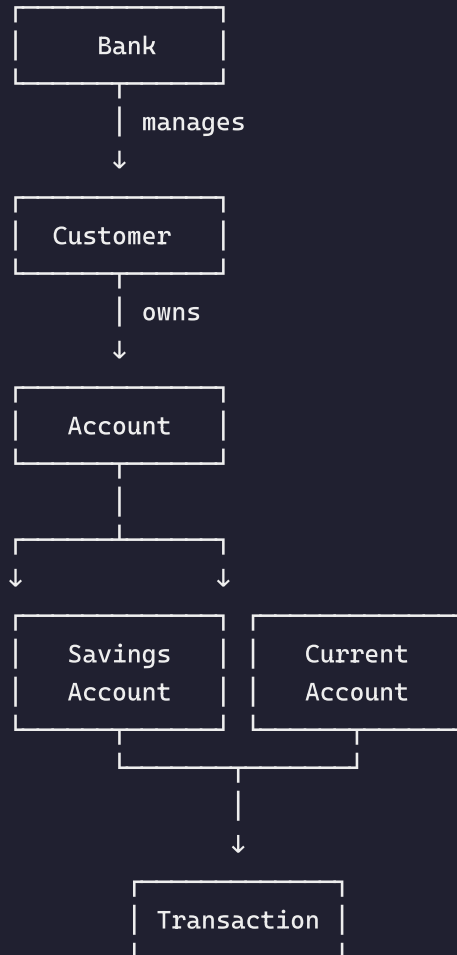
Object-Oriented Analysis (OOA) identifies the main objects/classes in a banking system and describes how they interact with each other.

Main Classes/Objects

1. **Customer** – Contains customer information such as name, ID, and address.
2. **Account** – Stores account information such as account number and balance.
3. **Savings Account** – A type of account that provides interest.
4. **Current Account** – A type of account used for regular transactions.

5. **Transaction** – Handles deposits, withdrawals, and transfers.
6. **Bank** – Manages customers, accounts, and transactions.
7. **Employee** – Performs banking operations and manages customer accounts.

Interconnection



Interconnection Explanation

- A Bank has many Customers.
- A Customer can have one or more Accounts.
- An Account can be a Savings Account or Current Account.
- Transactions are performed through an Account.
- Employees manage customers, accounts, and banking operations.

In short: OOA breaks the banking software into related objects such as Bank, Customer, Account, and Transaction, making the system easier to understand, develop, and maintain.

8. Differentiate among local variable, instance variable and static variable.

Local Variable	Instance Variable	Static Variable
Declared inside a method or block.	Declared inside a class but outside methods.	Declared inside a class using <code>static</code> .
Used only within that method/block.	Each object has its own copy.	Only one shared copy exists for the class.
Created when the method runs.	Created when an object is created.	Created when the class is loaded.
Must be initialized before use.	Gets a default value if not initialized.	Gets a default value if not initialized.
Example: <code>int x = 10;</code>	Example: <code>int age = 20;</code>	Example: <code>static int count = 0;</code>